When a Detector View Contains Its Label: A Reproducibility Audit of a Synthetic Shadow AI Dataset

Ezekiel Ologunde

Abstract

Shadow AI detection requires distinguishing observable AI use from organizational authorization and harmful activity. We audit one public synthetic enterprise generator at an immutable source revision. Two unchanged local executions each produce 191,370 sessions with 43 injected violations. Although the detector-view exporter removes the explicit violation label, it retains a categorical field set to a violation-specific value by the injection code. Equality testing on that field recovers all 43 labels without error; the original detector-view validator accepts the exported schema. This is label leakage, not successful anomaly detection. An explicit feature projection excludes the confirmed alias and separates join identities; five boundary checks pass and all rows in both runs are projected without modifying the originals. Raw exports differ because of identifiers, while canonical non-identifier observations and labels agree. The contribution is a revision-specific, reproducible benchmark audit and mitigation example, not a new discovery algorithm, proof of exhaustive leakage removal, or estimate of real enterprise Shadow AI prevalence.

Keywords

Shadow AI, benchmark validity, label leakage, reproducibility, AI inventory

1 Introduction and Scope

Discovering an AI dependency, contacting an AI service, violating an organizational approval policy, and disclosing sensitive information are different outcomes. Source scanners already inventory AI components: Trusera AI-BOM and Cisco AIBOM provide substantial discovery functionality [2, 3]. A new inventory command alone would not establish research novelty. Evaluations also need to establish that detector inputs do not encode the answer.

The ShadowAI-Enterprise source repository advertises separate detector, investigator and ground-truth views [1]. We inspect that separation and reproduce its generation pipeline. Our question is narrow: does the detector export at the inspected revision prevent a synthetic violation label from entering its features, and what does repeated execution actually reproduce? The motivation is consistent with established warnings about experimental pitfalls in security machine learning [4]; neither label leakage nor schema validation is a new concept.

Our contributions are an artifact-specific source-to-output trace of one direct label alias, two preserved local executions, a comparison against two transparent controls, and a separately authored feature contract. The targeted search did not locate a prior report of this exact mechanism in this source revision. That search result is not a priority guarantee. This technical note is suitable for author review; its single-artifact scope may be insufficient for a standalone research venue without a broader audit.

2 Source and Reproduction Protocol

We pin `Deepa-15/shadowai-enterprise-dataset` at commit `b1d29669`.... Acquisition records immutable URLs, byte sizes and SHA-256 for source/configuration files. No CSV records were present in the inspected Git tree. Consequently, the observations below come from locally executing the public generator, not from downloaded natural enterprise telemetry or an independently observed workplace.

The source imports data-generation and file/logging libraries. We run it in two separately copied working directories, preserving all downloaded source bytes. The unchanged seed is 42. The environment is Python 3.12, pandas 3.0.1, NumPy 2.3.5 and Faker 37.0.0. These satisfy the author's broad lower-bound requirements, but do not reproduce an exact original author dependency lock. Both pipeline processes exit successfully and produce ten output files each.

The source review first identified the candidate alias and an unseeded UUID source. A local audit protocol was then frozen before full execution; it is an exploratory mechanism-confirmation protocol, not an external preregistration or blind test. Each execution's detector export is joined one-to-one to its ground truth on `session_id`. We retain exact confusion counts rather than treating correlated synthetic rows as independent population samples. The two executions are a repeatability check, not two independent datasets.

3 Leakage Mechanism and Results

The injection function `apply_metadata` writes both `is_violation=True` and `session_type=shadow_ai_violation`. The function `export_detector_view` removes the first field and other investigator fields but does not remove `session_type`. The corresponding validator's forbidden-column list also omits this alias. These observations are tied to the pinned revision rather than all versions of the repository.

Both executions produce 191,370 records: 111,817 normal, 79,510 normal AI-use and 43 injected-violation sessions. The positive fraction is 0.02247%. The three controls in Table 1 use no trained model. The direct-alias control predicts a violation exactly when `session_type` equals the injected category. The domain control predicts a violation for membership in the five AI-service domains declared by the source configuration. The always-negative control predicts none.

The label-alias rule attains perfect agreement by reading an answer-bearing field. It demonstrates an input-isolation failure; it establishes no anomaly-detection capability. The original `verify_detector_view` function, extracted without module-level side effects and executed on each full detector frame, returns success. The source pipeline also completes its normal verification path.

Table 1: Per-execution results on the same synthetic 191,370 sessions, with 43 positives. Both executions agree. P and R denote precision and recall; "–" denotes undefined precision. The first row is an invalid shortcut, not detector performance.

Rule	TP	FP	FN	P	R
Direct label alias	43	0	0	1.000000	1.000
Always negative	0	0	43	–	0.000
AI-domain membership	43	79510	0	0.000541	1.000

The always-negative rule reaches 99.9775% accuracy with zero recall, illustrating why headline accuracy is uninformative at this constructed prevalence. Domain membership obtains full recall but 79,510 false positives and precision 0.000541. It detects service use rather than the injected violation outcome. We do not interpret these synthetic labels as independently verified organizational authorization decisions or evidence of actual data exfiltration.

4 Mitigation and Repeatability

We implement a separate explicit feature contract. It projects reviewed fields, keeps session and employee identifiers in a distinct join-metadata namespace, excludes the confirmed label alias, and rejects unexpected source columns. It also excludes `minutes_since_resource`; its provenance would require review before a strict network-only feature claim. This projection does not modify the downloaded source or original generated exports.

Five boundary checks cover rejection of the original leaked schema, an unexpected renamed alias, identity inclusion as a feature, missing required fields and accepted projection. All pass. All 191,370 rows from each execution pass projection, with original detector-file hashes unchanged. This is a mitigation for the declared schema, not proof that every remaining field is observable in real network telemetry, free from spurious correlation, or suitable for a deployment model. We do not train a model on the projected features or claim improved detection accuracy.

Fresh detector CSV hashes differ. The generator's `uuid.uuid4()` identifiers are not controlled by its numerical random seeds. However, canonical multisets of detector rows, after removing session IDs and ignoring row order, agree exactly. The same comparison with the violation label retained also agrees. Thus these executions reproduce substantive values under the stated environment, while not producing identical raw files. It would be misleading to describe identifier differences as different attack distributions.

5 Validity, Availability and Next Study

This is one source artifact and one revision. The direct alias was found by source inspection; its measured shortcut is a transparent mechanism confirmation. It does not establish the frequency of leakage across Shadow AI datasets or invalidate any particular published model result, because no such result was reproduced. No natural employee data, monitoring, paid model calls or third-party service probing are involved. Code review and analysis were AI-assisted; no independent human adjudication or maintainer confirmation is claimed.

Public availability is not a blanket redistribution license. The inspected dataset repository has no recognized license in its GitHub metadata and no root license file was acquired. We retain original source/generated exports locally and prepare an authored-code/results package with acquisition instructions, rather than assigning new terms to third-party material. A public artifact archive has not yet been released.

The immediate practical action is to prevent the confirmed alias from entering a model's feature view and to preserve identifiers only for joins or grouped splits. A stronger research paper would audit additional independently developed datasets, test cross-employee and temporal splits on justified features, and compare authorization labels against explicit policy/account evidence. Policy-aware runtime systems already exist, so adding a policy join would need its own measured incremental contribution.

6 Conclusion

The inspected exporter exposes an injected violation label through a second field, and its validator accepts the result. Two full local runs confirm the mechanism, while distinguishing byte-level identifier differences from substantive repeatability. An explicit projection removes the confirmed alias and enforces a bounded feature contract. The result supports a reproducible technical audit note, not a general Shadow AI detection or field-wide novelty claim.

References

[1] Deepa-15. 2026. ShadowAI-Enterprise Dataset. Inspected commit `b1d2966939edde18edc08a08f52145fc1378d567`. https://github.com/Deepa-15/shadowai-enterprise-dataset/tree/b1d2966939edde18edc08a08f52145fc1378d567

[2] Trusera. 2026. AI-BOM. Inspected commit `012d53d3`…. https://github.com/Trusera/ai-bom

[3] Cisco AI Defense. 2026. AI BOM. Inspected commit `1ee6f17c`…. https://github.com/cisco-ai-defense/aibom

[4] Daniel Arp, Erwin Quiring, Feargus Pendlebury, Alexander Warnecke, Fabio Pierazzi, Christian Wressnegger, Lorenzo Cavallaro, and Konrad Rieck. 2022. Dos and Don'ts of Machine Learning in Computer Security. In *USENIX Security*. https://www.usenix.org/conference/usenixsecurity22/presentation/arp